

Chapitre 2 : Conception et analyse des besoins

Introduction

Ce chapitre a pour objectif de présenter la conception globale du système ainsi que l'analyse des besoins nécessaires à sa réalisation. Il permet de définir les fonctionnalités attendues, les contraintes techniques et les choix architecturaux adoptés. Cette étape est essentielle pour garantir que le système développé répond efficacement aux objectifs fixés, notamment l'inspection automatique des pièces et le contrôle intelligent du bras robotique.

1.Présentation globale du système

1.1. Description du système de bras robotique intelligent

Le système proposé est un bras robotique intelligent équipé d'un module de vision permettant d'inspecter des objets en temps réel. Il repose sur une architecture distribuée combinant des composants matériels (ESP32, Raspberry Pi, servomoteurs) et logiciels (applications web et mobile, services backend).

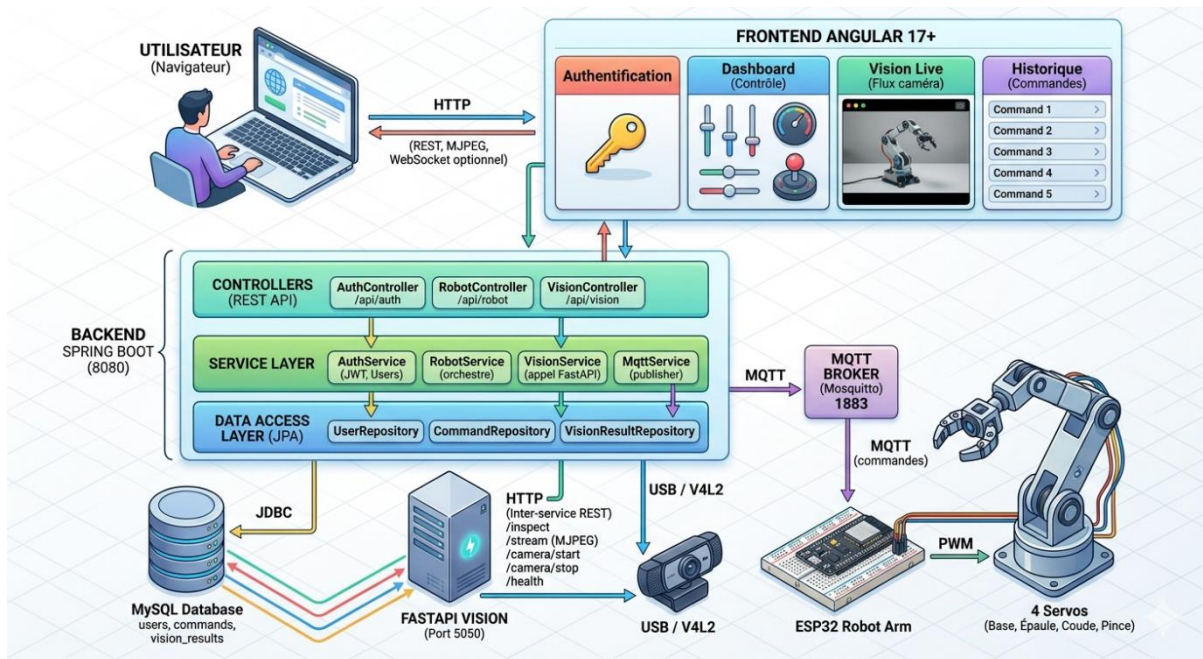


Figure 1. Vue d'ensemble de l'architecture (matérielle, logicielle, communication)

2.2. Principe de fonctionnement général

Le fonctionnement général consiste à capturer des images via une caméra, analyser ces images pour détecter les défauts, puis envoyer des commandes au bras robotique afin d'effectuer une action spécifique, comme le tri ou le déplacement des objets

2. Identification des besoins

2.1 Besoins fonctionnels

Le système doit permettre les fonctionnalités suivantes :

- **Acquisition vidéo en temps réel** : capturer un flux continu depuis la caméra USB ELP 8MP branchée sur le Raspberry Pi 4.
- **Inspection automatique** : détecter les défauts de positionnement des ressorts par template matching et NMS, et retourner un résultat PASS/FAIL.
- **Commande du bras 4-DOF** : piloter les quatre servomoteurs SG90 via des signaux PWM générés par l'ESP32.
- **Pick automatique** : en cas de défaut détecté, envoyer automatiquement une commande de séquence pick-and-place au bras.
- **Interface de supervision web** : visualiser le flux caméra (MJPEG), déclencher un scan, consulter le journal d'inspection depuis un navigateur.
- **Sauvegarde des présets** : enregistrer une image de référence en base de données MySQL pour les calibrages ultérieurs.
- **Contrôle manuel du bras** : depuis le tableau de bord, l'opérateur peut piloter manuellement chaque servomoteur (base, épaule, coude, pince).

2.2 Besoins non fonctionnels

Exigence	Description
Temps réel	Latence de détection < 2 secondes, streaming vidéo à 30 fps.
Fiabilité	Taux de faux positifs < 5% sur la détection de défauts.
Modularité	Architecture distribuée : chaque couche (vision, coordination, exécution) est indépendante et remplaçable.

Sécurité	Authentification JWT sur l'API Spring Boot. Connexions CORS contrôlées.
Portabilité	Interface Angular accessible depuis tout navigateur sur le réseau local.
Maintenabilité	Code structuré en services découplés (VisionService, RobotService, FastAPI).
Extensibilité	Possibilité d'ajouter d'autres capteurs ou caméras sans refonte de l'architecture.

Tableau 1. Les besoins non fonctionnelles

3. Étude du Bras Manipulateur

3.1 Choix du bras manipulateur

La démocratisation de la robotique a rendu les équipements bien plus accessibles qu'auparavant. Pour ce projet, plutôt que de concevoir une structure de zéro, nous avons choisi un bras manipulateur en kit (squelette en pièces détachées). Ce choix nous permet de nous concentrer sur le traitement d'image et l'intégration système, tout en bénéficiant d'une base mécanique fiable et économique.

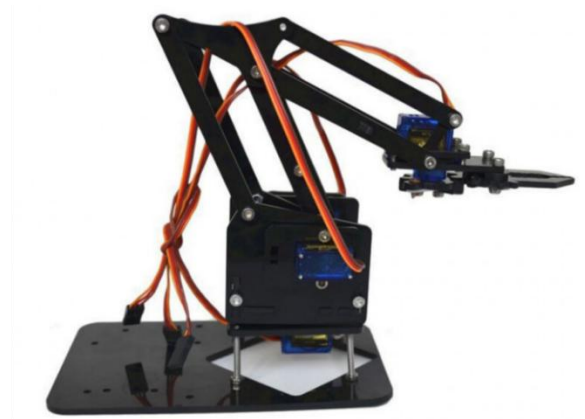
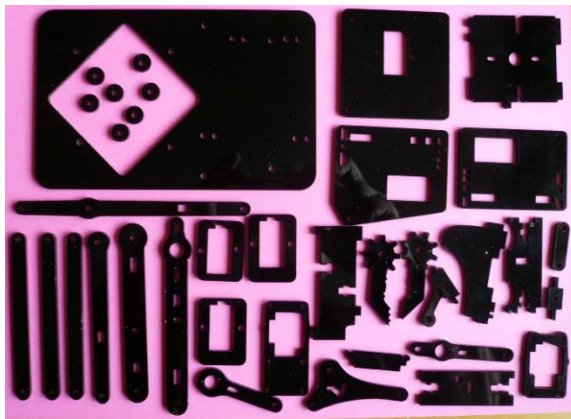


Figure 2. les composants et la bra complète du robot

3.2 Présentation du bras manipulateur

Le bras manipulateur est un outil programmable qui imite le mouvement d'un bras humain. Il est composé de plusieurs pièces reliées par des articulations (des axes). Ces articulations

permettent au robot de tourner ou de se déplacer en ligne droite pour atteindre des objets avec précision.

3.3 Constitution du bras manipulateur

Définition de la base

La base est la partie fixe du robot. C'est elle qui soutient tout le poids du bras et de l'objet transporté pour que l'ensemble reste bien stable. Elle est solidement assemblée avec des vis et des boulons. À l'intérieur, un servomoteur lui permet de pivoter sur elle-même jusqu'à 180°.

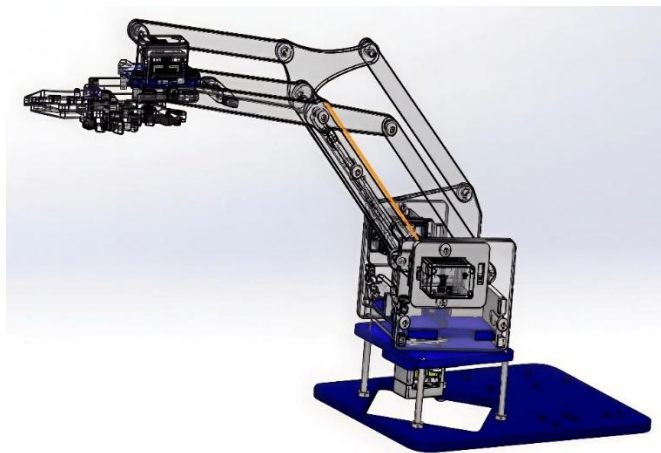


Figure 3. La base du robot

Définition du porteur

Le porteur est le corps principal du robot. Son rôle est de déplacer la pince exactement là où on en a besoin. Il fonctionne comme un bras humain : il est composé de segments rigides (les « os ») et d'articulations (les « coudes » ou « épaules »). Ce sont ces articulations, actionnées par des moteurs, qui permettent au bras de s'étendre, de monter ou de descendre.

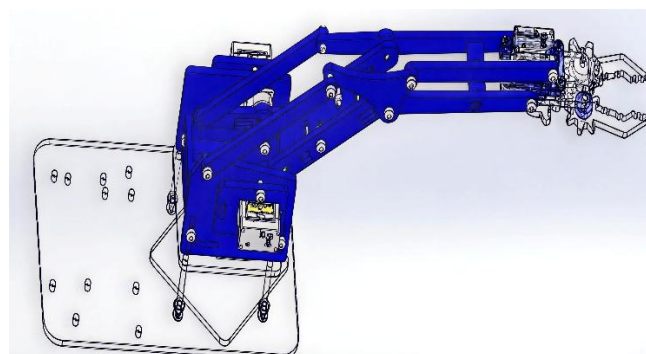


Figure 4 .La bra du robot

Définition de la pince

La pince est la « main » du robot, située tout au bout du bras. C'est elle qui permet d'attraper et de tenir les objets. Elle fonctionne grâce à un petit moteur (servomoteur) et un système d'engrenages qui ouvrent ou ferment les doigts de la pince selon les besoins.

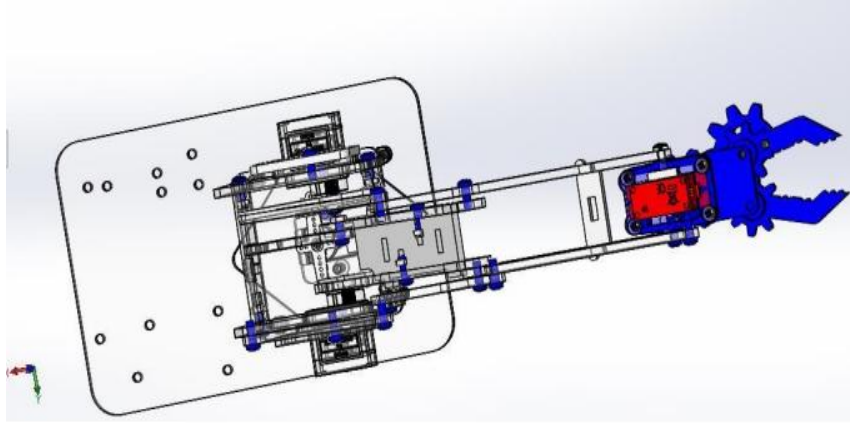


Figure 5. le pince du robot

3. Spécifications techniques

3.1 Architecture matérielle

L'architecture matérielle du système repose sur plusieurs composants connectés entre eux pour assurer le fonctionnement global. (en train de faire)

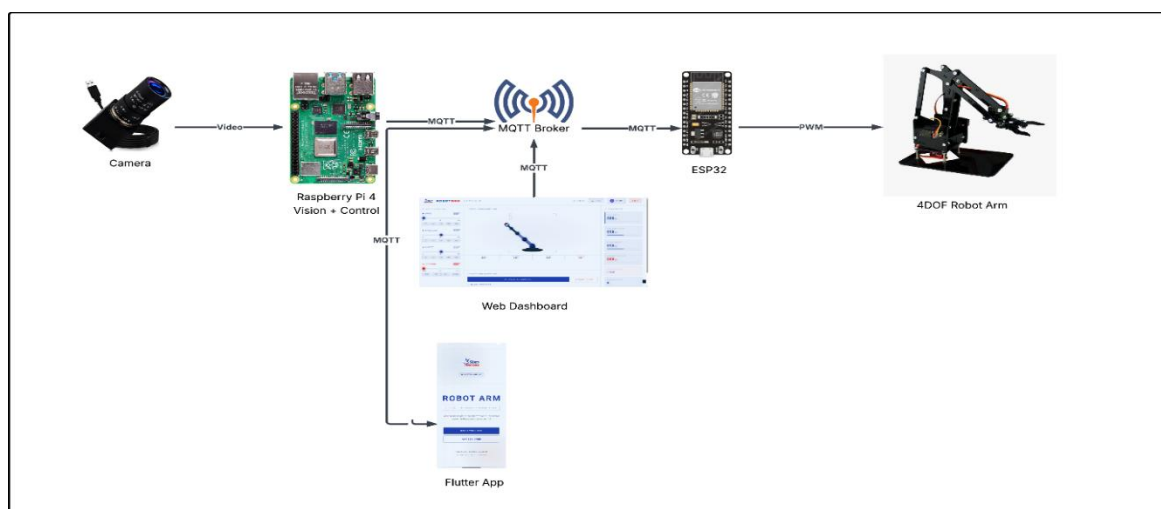


Figure 6. Architecture matérielle générale

Niveau	Composant principal	Rôle
Niveau Perception	Raspberry Pi 4	Acquisition vidéo (caméra ELP 8MP), traitement OpenCV, détection de défauts, calcul de cinématique, publication MQTT et streaming MJPEG via FastAPI.
Niveau Coordination	Broker MQTT + Spring Boot	Le broker Mosquitto distribue les messages entre le RPi et l'ESP32. Spring Boot orchestre la logique métier, l'authentification, l'API REST et la persistance MySQL.
Niveau Exécution	ESP32	Réception des angles via MQTT, génération des signaux PWM et pilotage des quatre servomoteurs SG90 du bras 4-DOF.

Tableau 2. Différentes niveau d'architecture matérielle

3.1.1.Composants Matériels du Système

Le choix des composants constitue une étape déterminante qui conditionne directement la précision du bras robotique et la fluidité de son contrôle. Nous avons sélectionné des technologies offrant un équilibre optimal entre puissance de calcul, réactivité en temps réel et robustesse industrielle.

➤ Raspberry Pi 4 Modèle B

Le Raspberry Pi 4 fait office de « cerveau » du système vision. Contrairement à un microcontrôleur standard, il possède la puissance nécessaire pour exécuter des algorithmes de vision artificielle complexes via **OpenCV**. Il assure l'acquisition vidéo, la détection des objets en temps réel et le calcul de la cinématique, tout en gérant la communication réseau haute performance.



Figure 7. Raspberry Pi 4 Modèle B

Spécification	Détail
Processeur	Broadcom BCM2711, Quad-core Cortex-A72 @ 1,8 GHz
RAM	4 Go LPDDR4 — suffisant pour OpenCV + FastAPI + MJPEG
Connectivité	Wi-Fi 802.11ac, Bluetooth 5.0, Gigabit Ethernet
Système	Raspberry Pi OS (Linux) — support natif Python, pip, uvicorn
Interface USB	4 ports USB 3.0 — connexion directe de la caméra ELP

Tableau 3. Caractéristique du carte Raspberry pi 4

➤ Caméra ELP 8MP

Pour garantir une détection précise, nous avons intégré une caméra **ELP (Modèle USB8MP02G-SFV)** de 8,0 Mégapixels. Son objectif **vari-focal (5–50 mm)** est un atout majeur : il permet d’ajuster manuellement le zoom et la mise au point pour adapter parfaitement le champ de vision à la zone de travail du robot.

- **Précision** : capteur haute définition pour isoler des objets avec une grande finesse.
- **Flexibilité** : objectif vari-focal 5–50 mm adapté à différentes distances de travail.
- **Interface** : USB 2.0 — plug-and-play sur Raspberry Pi (cv2.VideoCapture).



Figure 8. Camera ELP 8M

➤ ESP32

L’ESP32 agit comme l’unité d’exécution du système. Il reçoit les commandes du broker Mosquitto via le protocole **MQTT**. Sa rapidité de traitement et sa gestion native du Wi-Fi en

font le candidat idéal pour générer les signaux **PWM** nécessaires au pilotage des moteurs sans latence perceptible, garantissant ainsi un mouvement fluide du bras.

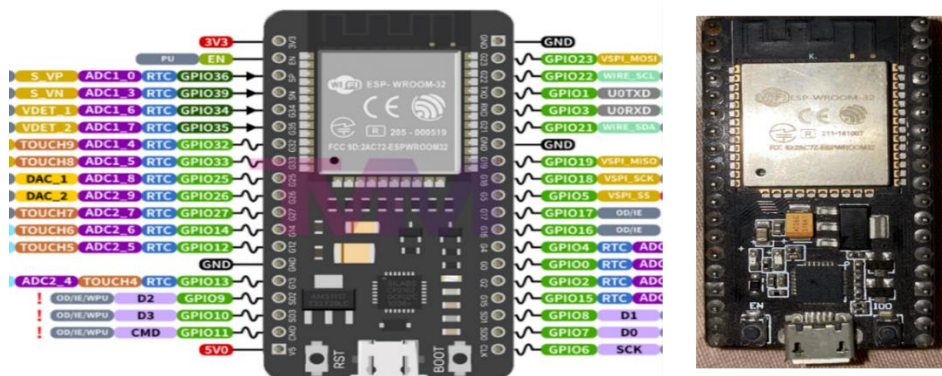


Figure 9. microcontrôleur ESP32

Spécification	Détail
CPU	Xtensa LX6 Dual-Core @ 240 MHz
Connectivité	Wi-Fi 802.11 b/g/n, Bluetooth 4.2 intégrés
GPIO	34 broches programmables — sorties PWM sur GPIO 13, 12, 14, 27
Protocole	MQTT via bibliothèque PubSubClient (Arduino)
Alimentation	3,3 V logique — alimenté par le bloc JC Power 5V

Tableau 4. Caractéristique de carte ESP32

➤ Servomoteurs TowerPro SG90

Le bras 4-DOF est animé par quatre servomoteurs TowerPro SG90. Ces micro-servomoteurs ont été choisis pour leur légèreté et leur capacité à maintenir une position angulaire précise sur 180°. Ils permettent au prototype de réaliser des manipulations délicates avec une consommation d'énergie maîtrisée.



Figure 10. Servo moteur SG90

Articulation	Plage angulaire
Articulation	Angle commandé par ESP32
Base (rotation)	0 – 180° — pivotement horizontal
Épaule (porteur)	0 – 180° — extension/rétraction
Coude (avant-bras)	0 – 180° — flexion/extension
Pince (gripper)	0 – 90° — ouverture/fermeture

Tableau 5. Caractéristique de Servo moteur SG90

➤ Driver module PCA9685

Le **PCA9685 16-channel 12-bit PWM driver** est un circuit intégré utilisé comme générateur de signaux PWM (Pulse Width Modulation). Il permet de contrôler jusqu'à 16 sorties indépendantes avec une résolution de 12 bits, via une communication I2C.

Ce module est particulièrement adapté aux applications nécessitant le pilotage simultané de plusieurs dispositifs, tels que les servomoteurs, les LEDs ou d'autres charges nécessitant un signal PWM précis. Il est souvent utilisé avec des microcontrôleurs comme Arduino Uno ou ESP32 pour étendre le nombre de sorties PWM disponibles.

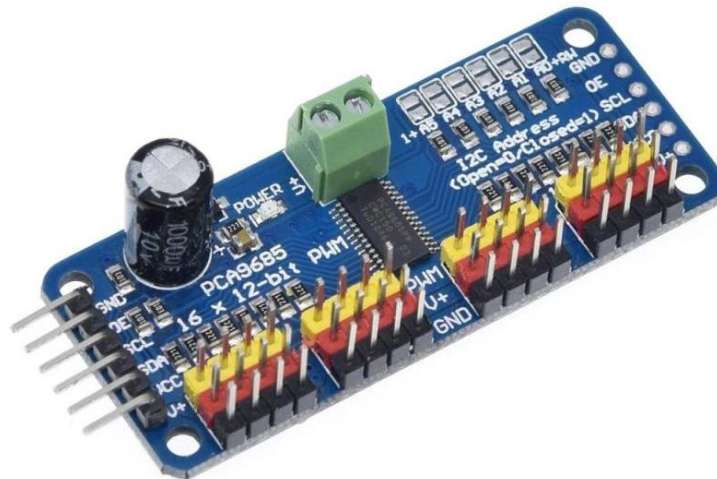


Figure 11. driver PCA9685

Caractéristique	Valeur / Description
Nombre de canaux	16 sorties PWM indépendantes
Résolution	12 bits (4096 niveaux)
Interface de communication	I2C
Adresse I2C	Configurable (jusqu'à 62 modules sur le même bus)
Fréquence PWM	Environ 40 Hz à 1000 Hz
Tension logique (VCC)	3.3V à 5V
Tension alimentation (V+)	5V à 6V (pour servomoteurs)
Courant par sortie	Faible (nécessite driver externe pour fortes charges)
Horloge interne	Oscillateur intégré
Compatibilité	Arduino, ESP32, Raspberry Pi, etc.
Type de commande	Numérique (via bus I2C)
Applications principales	Servomoteurs, LEDs, robotique, automatisation

Tableau 6. Caractéristique du driver PCA9685

➤ JC Power 25-5

Pour alimenter l'ensemble, nous utilisons un bloc **JC Power (Modèle 25-5)**, délivrant une tension constante de **5 V avec un courant de 5 A (25 W)**. Cette source d'énergie industrielle est indispensable pour supporter la consommation combinée du Raspberry Pi 4 et l'appel de courant des quatre servomoteurs lors de leurs mouvements simultanés.



Figure 12. JC Power (Modèle 25-5)

Composant	Consommation
Raspberry Pi 4	~ 3,0 A (charge pleine)
4 x SG90 (pic)	~ 4 x 0,36 A = 1,44 A
ESP32 + L298N	~ 0,3 A
Total estimé	~ 4,74 A → 5 A requis (JC Power 25-5)

Tableau 7. Details de consommation sur JC Power (Modèle 25-5)

4. Architecture logicielle

L'architecture logicielle du système repose sur une organisation modulaire permettant une séparation claire des responsabilités. Elle comprend un backend développé avec Spring Boot assurant la gestion des données et des services, ainsi qu'un service FastAPI en Python dédié au traitement et à l'analyse des images. L'interaction avec l'utilisateur est assurée via une interface web réalisée avec Angular et une application mobile développée avec Flutter, offrant ainsi une supervision et un contrôle du système à distance.

4.1 . Vue d'ensemble

L'architecture logicielle suit un modèle en couches adapté aux contraintes IoT et embarquées

Couche	Responsabilité
Couche IHM	Angular (TypeScript) — dashboard web SPA, streaming MJPEG, contrôles en temps réel.
Couche Service	Spring Boot (Java) — API REST, sécurité JWT, orchestration, persistance MySQL.
Couche Vision	Python FastAPI (RPi 4) — OpenCV, template matching, NMS, MJPEG stream.
Couche Communication	MQTT Mosquitto — publish/subscribe entre RPi, Spring Boot et ESP32.
Couche Embarquée	Firmware ESP32 (Arduino C++) — abonnement MQTT, génération PWM, pilotage SG90.

Tableau 8. Tableau explicatif de différent couche du systeme

4.2. Backend

➤ Spring Boot

Spring Boot joue le rôle d'**orchestrateur central**. Il expose l'API REST consommée par Angular, délègue les opérations de vision au service FastAPI (via RestTemplate HTTP), gère la persistance MySQL (JPA/Hibernate), et publie les commandes d'angles sur le broker MQTT via MqttPublishService.



Figure 13. logo du spring boot

➤ FastAPI

FastAPI est un framework web moderne et ultra-performant pour la création d'API REST avec Python 3.6+, basé sur les standards ouverts (OpenAPI, JSON Schema). Il est reconnu pour sa grande vitesse (proche de Node.js/Go), facilitant un développement rapide avec peu de bugs.



Figure 14. logo du fastAPI

Le service FastAPI expose cinq endpoints REST :

Endpoint	Rôle
POST /camera/start	Ouvre le flux cv2.VideoCapture(2) sur la caméra ELP.
POST /camera/stop	Libère la caméra.
POST /inspect	Reçoit une frame base64, exécute le pipeline inspection, retourne { isDefect, confidence, defectType }.
GET /stream	Flux MJPEG boundary-encoded à 30 fps — pointé directement par le tag Angular.
GET /health	Contrôle de disponibilité (utilisé par Spring Boot).

Tableau 9. Endpoint du service FastAPI

4.3. Frontend

Le frontend du système est assuré par deux interfaces complémentaires développées avec Angular et Flutter. L'application web Angular a été développée sur vs code offre une plateforme de supervision accessible via navigateur, permettant de visualiser les données, contrôler le bras robotique et suivre les résultats d'analyse en temps réel.

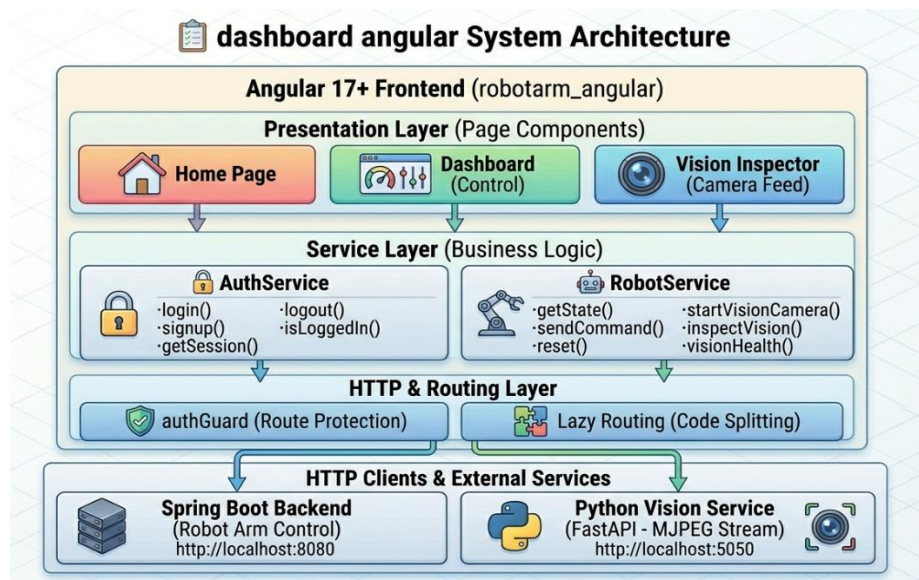


Figure 17. Architecture du Dashboard

➤ Angular

Angular est un **framework frontend open-source** basé sur TypeScript, maintenu par Google et la communauté, utilisé pour créer des applications web "monopages" (SPA) performantes et évolutives. Il permet de concevoir des interfaces riches et dynamiques, idéales pour les applications d'entreprise.



Figure 15. logo du angular

➤ Flutter

Flutter est un framework open source de Google permettant de créer des applications natives compilées pour mobile (iOS, Android), web et desktop à partir d'une base de code unique. Basé

sur le langage Dart, il utilise une approche "tout est widget" pour concevoir des interfaces graphiques rapides et fluides



Figure 16. logo Flutter

l'application mobile Flutter fournit une solution portable et intuitive, facilitant l'accès au système à distance

4.5. Protocoles de communication IoT

Les protocoles de communication IoT jouent un rôle essentiel dans l'échange de données entre les différents composants du système. Dans ce projet, plusieurs protocoles sont utilisés afin d'assurer une communication fiable et en temps réel. Le protocole MQTT est privilégié pour la transmission des messages entre les dispositifs, grâce à sa légèreté et son efficacité dans les environnements IoT. Le protocole HTTP est utilisé pour les échanges entre les services backend et les interfaces utilisateur. Enfin, le flux MJPEG permet la diffusion en continu des images capturées par la caméra, assurant ainsi une visualisation en temps réel.

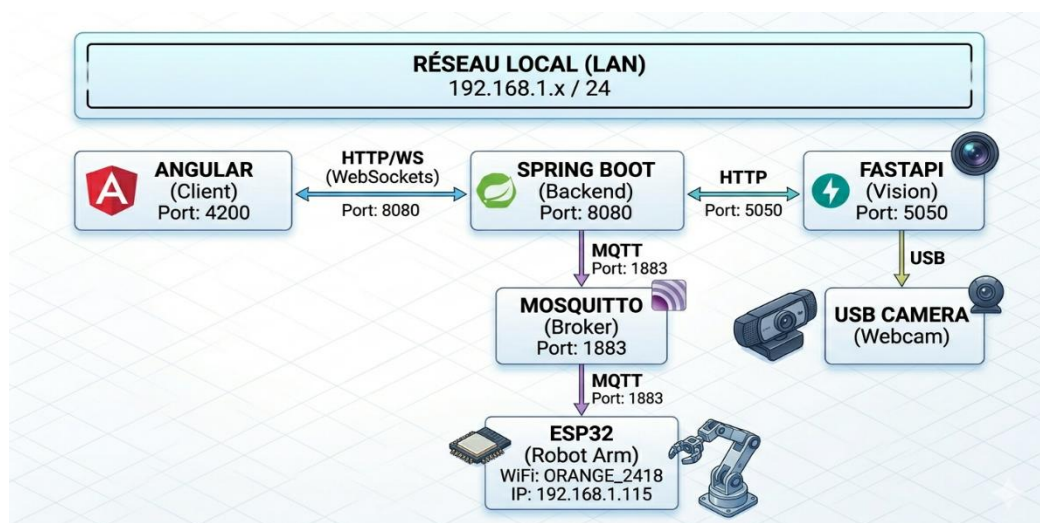


Figure 18. Architecture de réseau local

Protocole	Trajet	Détail
MQTT	Spring Boot ↔ ESP32	Commandes d'angles (robot/arm/angles), statuts (robot/arm/status). QoS 0, broker Mosquitto local.
HTTP/REST	Angular → Spring Boot	API JSON sur le port 8080. Authentification JWT Bearer token.
HTTP/REST	Spring Boot → FastAPI	Appels synchrones sur localhost:5050 pour démarrer la caméra et lancer l'inspection.
MJPEG	FastAPI → Angular	Flux multipart/x-mixed-replace direct sur GET /stream (port 5050). Latence < 100 ms.
Serial/PWM	ESP32 → SG90	Signaux PWM 50 Hz, largeur d'impulsion 0,5–2,4 ms correspondant à 0–180°.

Tableau 10. Les communications IOT utilisés

4.6. Flux de données entre composants

Le cycle de fonctionnement du robot suit un cheminement précis à travers les trois niveaux :

1. **Capture vidéo — RPi 4 + caméra ELP** : Le Raspberry Pi 4 capture le flux en continu. OpenCV traite chaque image pour détecter la position des ressorts par template matching.
2. **Inspection et décision — FastAPI Python** : Le service vision analyse la frame, applique le NMS, calcule les distances en mm et retourne un résultat PASS/FAIL avec la liste des ressorts défectueux.
3. **Coordination — Spring Boot + Broker MQTT** : Le backend reçoit le résultat via HTTP, le pousse vers Angular par WebSocket, et si défaut : publie la commande d'angles sur le topic MQTT robot/arm/angles.
4. **Exécution — ESP32** : L'ESP32 s'abonne au topic, reçoit les angles (base, épaule, coude, pince) et génère les signaux PWM correspondants pour actionner le bras.
5. **Retour position home** : Après le pick, le bras est renvoyé à la position d'origine (90°, 90°, 90°, 0°) automatiquement.

5.Modélisation du système

5.1. Identification des acteurs

Ce tableau résume les différents acteurs leur type sa description ainsi que leur rôle principal

Acteur	Type	Description	Rôle principal
Utilisateur	Humain	Personne qui utilise le système via une interface web ou mobile	Superviser, contrôler le système et consulter les résultats
Système de vision	Système	Module de traitement d'images basé sur la caméra et les algorithmes	Capturer, analyser les images et détecter les défauts
Bras robotique	Système	Dispositif mécanique composé de servomoteurs	Exécuter les actions physiques (pick-and-place)
ESP32	Matériel	Microcontrôleur chargé de piloter les servomoteurs	Recevoir les commandes et contrôler les mouvements du bras
Raspberry Pi	Matériel	Unité de traitement principale	Gérer le traitement des images et la coordination du système
Caméra	Capteur	Dispositif de capture d'images	Fournir le flux vidéo pour l'analyse
Interface Web	Logiciel	Application développée avec Angular	Permettre la supervision et le contrôle via navigateur
Application Mobile	Logiciel	Application développée avec Flutter	Permettre l'accès et le contrôle à distance
Backend	Logiciel	Serveur (Spring Boot, FastAPI)	Gérer les données, la logique métier et la communication entre les modules

Tableau 11. Identification des acteurs

5.2. Diagramme de cas d'utilisation

Ce diagramme de cas d'utilisation UML représente les interactions entre les différents acteurs et le système de contrôle du bras robotique intelligent.

L'acteur principal est l'utilisateur, qui interagit avec le système à travers plusieurs fonctionnalités. Il peut lancer une inspection, visualiser le flux vidéo en temps réel, contrôler le bras robotique et consulter les résultats de l'analyse. Ces actions traduisent les besoins fonctionnels principaux du système.

La caméra joue un rôle essentiel en capturant les images des objets à analyser. Ces images sont ensuite transmises au système de vision, qui se charge de les traiter. Ce module effectue l'analyse des images afin de détecter d'éventuels défauts ou d'identifier les objets.

Une fois l'analyse terminée, le système prend une décision et envoie des commandes au bras robotique. Ce dernier exécute alors les actions nécessaires, comme le déplacement ou le tri des objets à travers une opération de type pick-and-place.

Ainsi, le diagramme met en évidence un enchaînement logique des opérations : acquisition des images, traitement et analyse, prise de décision, puis exécution physique. Il montre également la collaboration entre les composants matériels et logiciels pour assurer le bon fonctionnement du système.

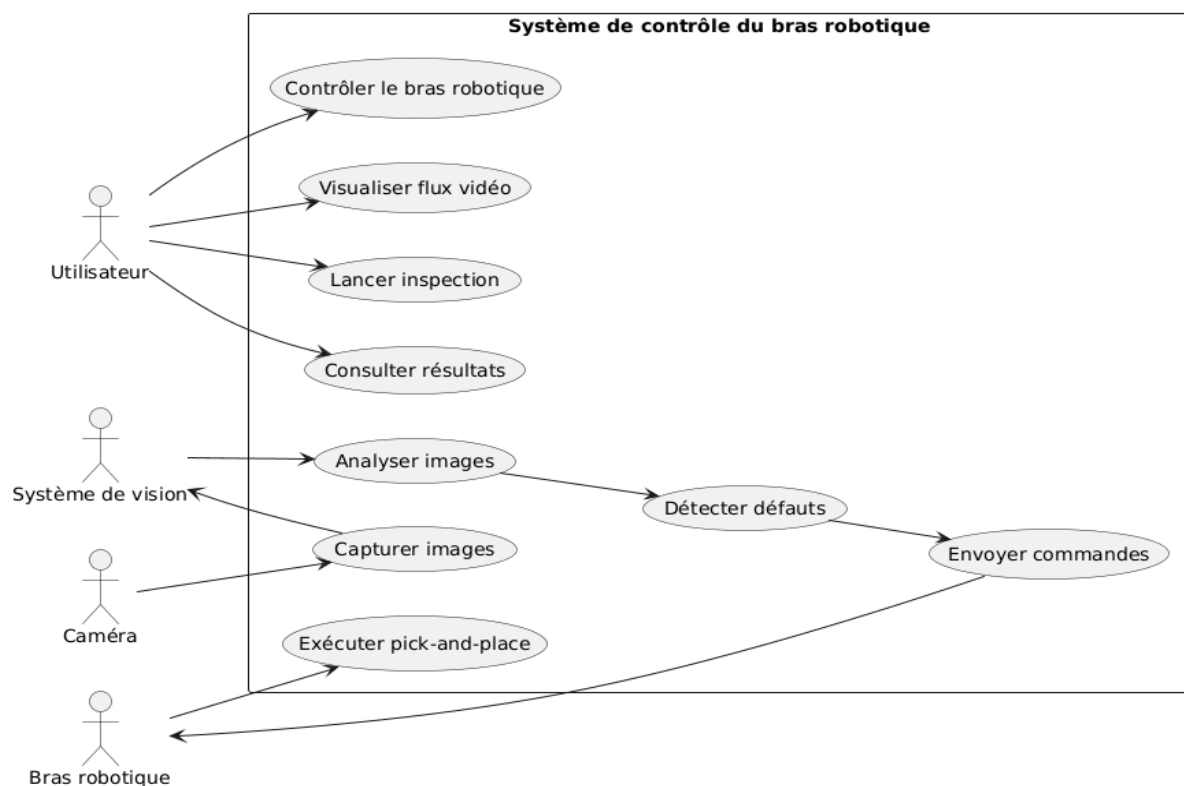


Figure 19. Diagramme use case UML

5.3. Diagramme de séquence

Ce diagramme illustre l'interaction temporelle entre les composants lors d'un cycle d'inspection :

1. L'opérateur clique « Scan Now » sur le dashboard Angular.
2. Angular envoie POST /api/robot/vision/inspect + frame base64 → Spring Boot.
3. Spring Boot transmet POST /inspect → FastAPI Python (RPi).
4. FastAPI exécute le pipeline : décodage JPEG → template matching → NMS → calcul distances.
5. FastAPI retourne { isDefect, confidence, defectType } → Spring Boot.
6. Spring Boot retourne le résultat → Angular (affichage PASS/FAIL).
7. [Si défaut] Spring Boot publie les angles de pick sur MQTT (robot/arm/angles).
8. ESP32 reçoit les angles, génère les PWM, le bras exécute le pick.
9. Spring Boot envoie la commande return-home après 1,5 s.

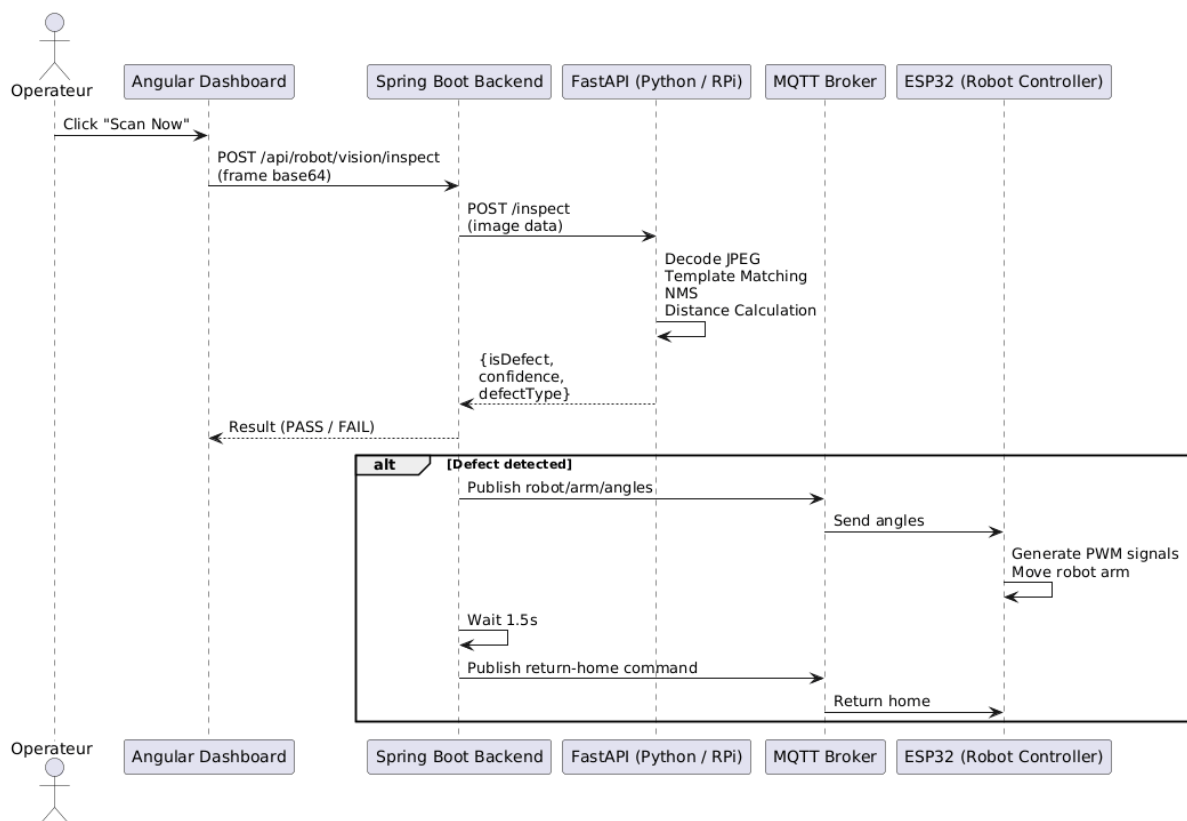


Figure 20. Diagramme de séquence UML

5.4.Diagramme d'états — Cycle de vie du bras robotique

Le bras traverse quatre états principaux :

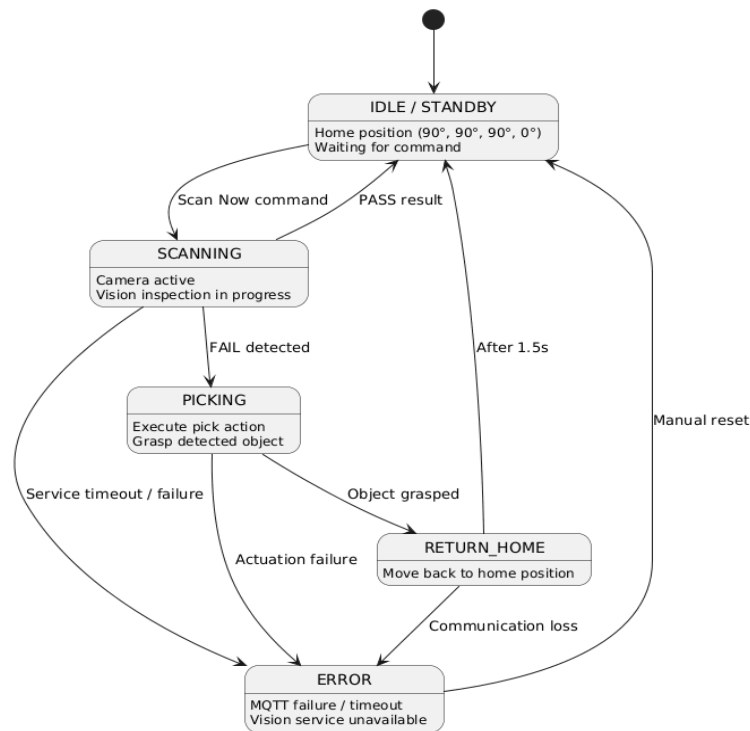


Figure 21. Diagramme d'états UM

État	Description
IDLE / STANDBY	État initial. Bras en position home (90°, 90°, 90°, 0°). Attente de commande.
SCANNING	Caméra active, inspection en cours. Transition vers IDLE (PASS) ou PICKING (FAIL).
PICKING	Commande pick en exécution. Transition vers RETURN_HOME après saisie de la pièce.
RETURN_HOME	Bras retournant à la position home. Transition vers IDLE après 1,5 s.
ERROR	État d'erreur (perte MQTT, timeout, service vision indisponible). Nécessite reset manuel.

Tableau 12. Description du diagramme d'état

5.5. Diagramme de déploiement

Le diagramme de déploiement cartographie les nœuds physiques et les artefacts déployés :

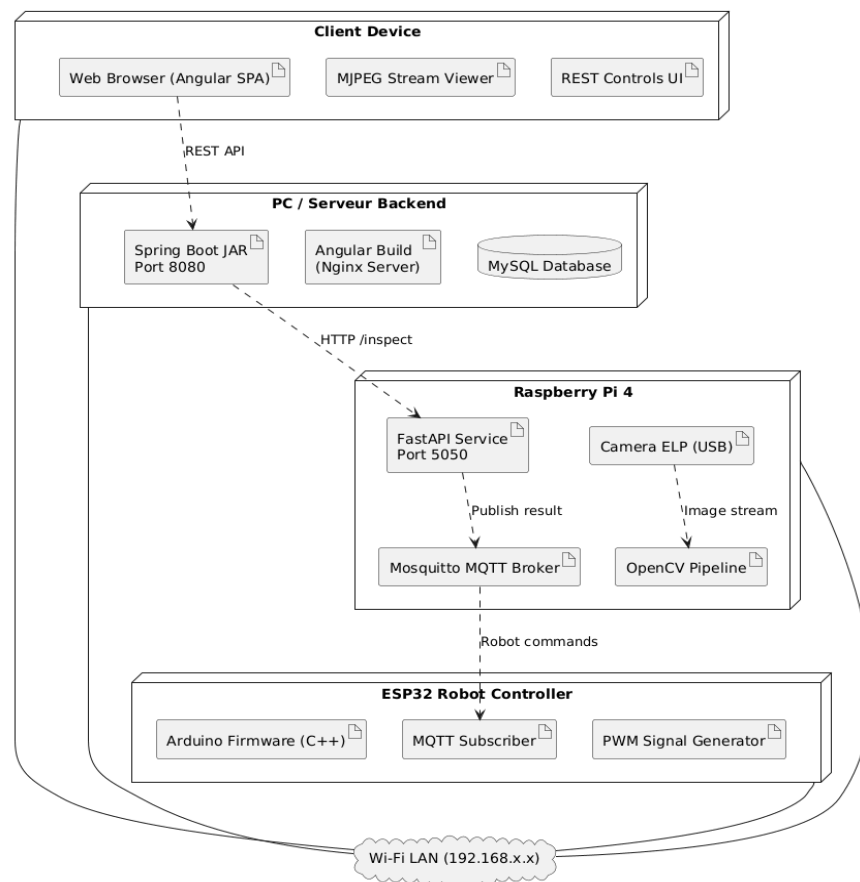


Figure 22. Diagramme de déploiement UML

Nœud / Réseau	Artefacts / Rôle
Nœud : PC / Serveur	Spring Boot JAR (port 8080), Angular build servi par Nginx, base MySQL.
Nœud : Raspberry Pi 4	Python FastAPI (port 5050), OpenCV, broker Mosquitto, caméra ELP (USB).
Nœud : ESP32	Firmware Arduino C++, abonnement MQTT, génération PWM.
Nœud : Navigateur	Application Angular SPA, streaming MJPEG, contrôles REST.
Réseau : Wi-Fi LAN	192.168.x.x — interconnecte tous les nœuds.

Tableau 13. Description du diagramme de déploiement

6. Pipeline de Vision Artificielle

6.1. Acquisition et prétraitement

L'acquisition est réalisée par **cv2.VideoCapture(2)** sur le Raspberry Pi 4. Chaque frame est convertie en niveaux de gris pour réduire la complexité de calcul. La résolution de travail est de 1920×1080 pixels, ce qui assure une précision suffisante pour le calcul des distances en mm via le facteur de conversion calibré (0,01009 mm/pixel).

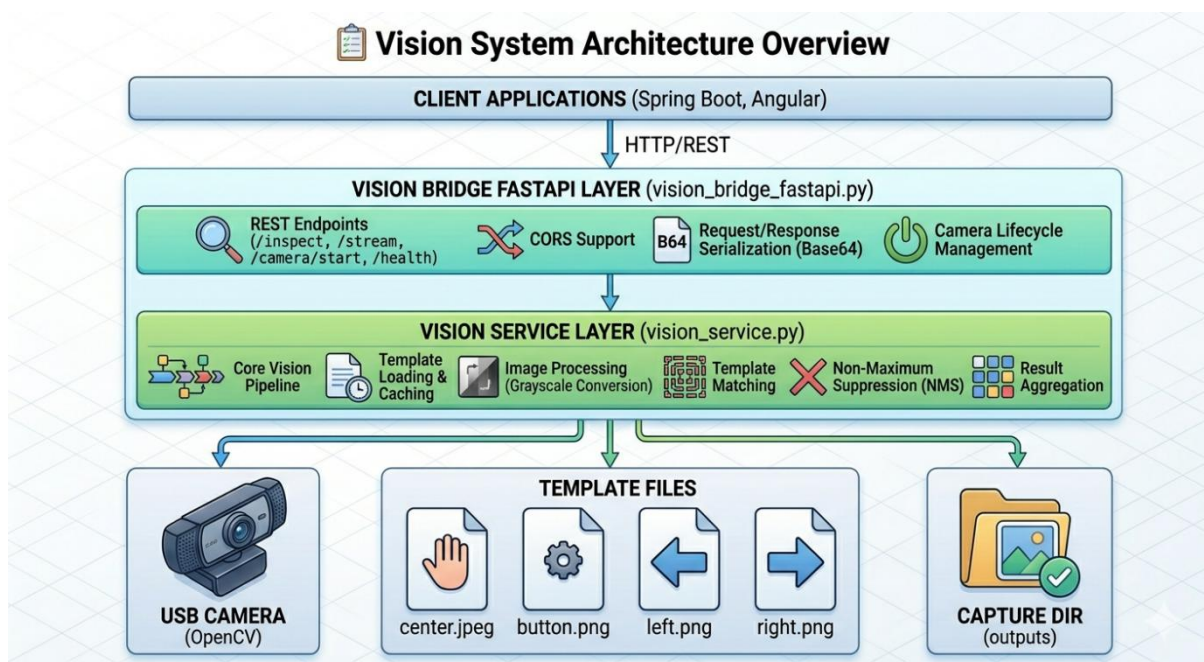


Figure 23 . Architecture de système de vision

6.2 Détection par template matching

Quatre templates sont utilisés pour localiser les éléments de référence dans l'image

Template	Rôle et seuil
center.jpeg	Centre du ressort — seuil 0,58 (haute précision requise)
button.png	Marqueur de référence bouton — seuil 0,66
left.png	Marqueur limite gauche — seuil 0,60
right.png	Marqueur limite droite — seuil 0,60

Tableau 14. Détails de Templates utilisés

La méthode utilisée est **cv2.TM_CCOEFF_NORMED**. Les correspondances au-dessus du seuil sont collectées, puis la **Non-Maximum Suppression (NMS)** élimine les détections redondantes (overlap > 0,30).

6.3. Logique de décision PASS/FAIL

Pour chaque centre de ressort détecté, deux distances sont calculées :

- **Distance Y** : écart vertical au marqueur bouton, converti en mm par le facteur de calibration.
- **Distance X** : écart horizontal au marqueur bouton, converti en mm.

Les distances sont comparées à des valeurs nominales selon deux plages :

Plage	Valeur nominale et tolérance
Plage 0–2 mm (Y)	Nominal : 1,70 mm $\pm$ 0,10 mm
Plage 2–5 mm (Y)	Nominal : 4,45 mm $\pm$ 0,10 mm
Plage 0–2 mm (X)	Nominal : 1,25 mm $\pm$ 0,10 mm
Plage 2–5 mm (X)	Nominal : 3,75 mm $\pm$ 0,10 mm

Tableau 15. Les valeur nominaux et tolérances

Si la distance mesurée s'écarte de plus de $\pm 0,10$ mm de la valeur nominale, le ressort est classé **FAIL** (rouge). Un rectangle rouge encadre le ressort défectueux et le signal de pick est envoyé au bras robotique. Si tous les ressorts sont dans la tolérance, le résultat est **PASS** (vert).

Conclusion

Ce chapitre a présenté l'analyse complète des besoins et la conception détaillée du système. L'**architecture distribuée en trois niveaux** répond aux contraintes de temps réel imposées par l'inspection industrielle. Les composants matériels ont été sélectionnés pour leur complémentarité et leur adéquation avec les besoins en termes de puissance de traitement, connectivité et précision de mouvement.

Les diagrammes UML formalisent les interactions entre composants. La chaîne vision et la chaîne de commande ont été conçues pour fonctionner de manière **synchronisée et automatique**, réduisant l'intervention humaine au strict minimum.

Le chapitre suivant détaillera la réalisation et l'implémentation de chaque composant du système.

Chapitre 2 : <i>Conception et analyse des besoins</i>	1
Introduction	1
1.Présentation globale du système	1
1.1. Description du système de bras robotique intelligent.....	1
2.2. Principe de fonctionnement général	2
2. Identification des besoins	2
2.1 Besoins fonctionnels.....	2
2.2 Besoins non fonctionnels.....	2
3. Étude du Bras Manipulateur.....	3
3.1 Choix du bras manipulateur.....	3
3.2 Présentation du bras manipulateur.....	3
3.3 Constitution du bras manipulateur.....	4
3. Spécifications techniques	5
3.1 Architecture matérielle	5
3.1.1.Composants Matériels du Système	6
4. Architecture logicielle	11
4.1 . Vue d'ensemble	11
4.2. Backend	12
4.3. Frontend.....	13
4.5. Protocoles de communication IoT	14
4.6. Flux de données entre composants	15
5.Modélisation du système	16
5.1. Identification des acteurs	16
5.2. Diagramme de cas d'utilisation	16
5.3. Diagramme de séquence	18
5.4.Diagramme d'états — Cycle de vie du bras robotique.....	19
5.5. Diagramme de déploiement.....	20

6. Pipeline de Vision Artificielle.....	21
6.1. Acquisition et prétraitement	21
6.2 Détection par template matching	21
6.3. Logique de décision PASS/FAIL	22
Conclusion.....	22